# Dynamic programming

Recursion splits a problem into subproblems. Dynamic programming also identifies repeated subproblems and solves each state once.

Read **states and dependencies → rod cutting → knapsack → longest common subsequences and alignment → matrix chains and optimal binary search trees**.

Prerequisites are recursion, asymptotic analysis, arrays, and trees. Connect the problem definition, recurrence, evaluation order, and reconstruction into one algorithm.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# Decide what a state means first

| Design question | What the answer must establish |
| --- | --- |
| Is the state sufficient? | Equal states have the same future choices and costs. |
| Is the recurrence complete? | Every legal optimal solution belongs to a candidate branch. |
| Which states come first? | Boundaries and dependencies determine a valid evaluation order. |
| What must be returned? | Values, stored choices, and reconstructed solutions serve different purposes. |

For each model, fix the legal solutions and cost before comparing time, space, and the ability to reconstruct an optimum.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Dynamic programming starts with state meaning

Dynamic programming (DP) merges repeated subproblems into states, stores their answers, and combines them to solve the original instance.

- **State meaning:** Which feasible solutions does one entry optimize over?
- **Recurrence and proof:** Do candidates cover every optimum, and are all candidates feasible?
- **Evaluation and reconstruction:** Solve dependencies first; record a choice attaining each optimum.

Optimal substructure justifies composition. Overlap explains the benefit of caching. Neither replaces a precise state definition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A state must retain everything that affects the future

Two histories may share a state only if they have the same feasible continuations, future cost rules, and remaining objective.

Past value can be accounted for separately. Information that changes future choices belongs in the state.

For rod cutting, prices depend only on piece length, cuts are free, and there is no piece-count limit. Every remainder of length $j$ has the same options, so $r[j]$ suffices.

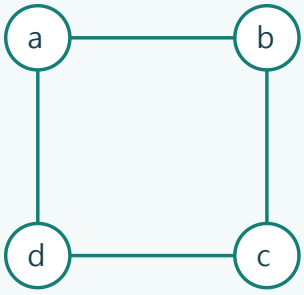
With an additional limit of $k$ pieces, remaining length alone is insufficient: retain the remaining piece allowance too.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Longest simple paths: endpoints omit a resource constraint

The undirected graph has edges $ab, bc, cd, da$, each of length 1. A simple path cannot repeat vertices.

### Same endpoints do not mean the same available vertices




Whole optimum: a → b → c → d

Suffix: c → d (1)

Independent longest: c → b → a → d (3)

Splice: a → b → c → b → a → d

Repeated a, b: infeasible

$a \to b \to c \to d$ is a longest simple path from $a$ to $d$, of length 3. Its suffix $c \to d$ is not longest between its endpoints: $c \to b \to a \to d$ has length 3. Substitution yields $a, b, c, b, a, d$, which is infeasible.

The endpoint state omits vertices consumed by the prefix; independently optimal subpaths need not compose.



## A richer state repairs correctness at exponential cost

Fix a start $s$. Define $D[S, v]$ as the longest simple path from $s$ to $v$ using exactly vertex set $S$; use $-\infty$ when none exists.

$$D[\{s\}, s] = 0, \qquad D[S, v] = \max_{\substack{u \in S \setminus \{v\} \\ uv \in E}} (D[S \setminus \{v\}, u] + 1).$$

Other base cases are $-\infty$. Evaluate nonbase states only for $s, v \in S$ and $v \neq s$. The answer is $\max_{S \ni s,t} D[S, t]$.

Set size strictly increases, giving a state DAG. Time is at most $O(2^{|V|}|V|^2)$: this is exponential DP. If the original graph is a DAG, longest paths can instead use endpoint states in topological order.

## Rod cutting: assumptions and state

Given integer length $n \geq 0$ and prices $p[1..n]$, cut the whole rod into positive integer lengths and sell every piece. Cuts are free; a length may occur repeatedly.

$$r[j] = \text{maximum revenue using exactly length } j, \qquad r[0] = 0.$$

Example prices, in revenue units:

| Length $i$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $p[i]$ | 1 | 5 | 8 | 9 |

Find $r[4]$ and the piece lengths. Selling the entire rod uncut is feasible.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Choose the first finished piece

Every nonempty decomposition has a first piece of length $i$, with $1 \leq i \leq j$. Sell that piece directly; only its remainder is subdivided further.

$$r[j] = \max_{1 \leq i \leq j} \{p[i] + r[j-i]\}, \qquad j \geq 1.$$

- $i = j$ represents no cut: $p[j] + r[0]$.
- Use $p[i]$, not $r[i]$: here $i$ denotes a finished piece.
- The remainder is strictly shorter, enabling induction on $j$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Prove both directions of the recurrence

Let $OPT(j)$ be the true optimum. Assume all shorter states are correct.

**Upper-bound direction:** An optimal decomposition starts with some $i$. Its remaining revenue is at most $r[j-i]$. Hence $OPT(j) \leq p[i] + r[j-i] \leq \max_i(p[i] + r[j-i])$.

**Lower-bound direction:** For every $i$, one piece of length $i$ followed by an optimal decomposition of length $j-i$ is feasible and uses exactly $j$. Thus $p[i] + r[j-i] \leq OPT(j)$; taking a maximum preserves the inequality.

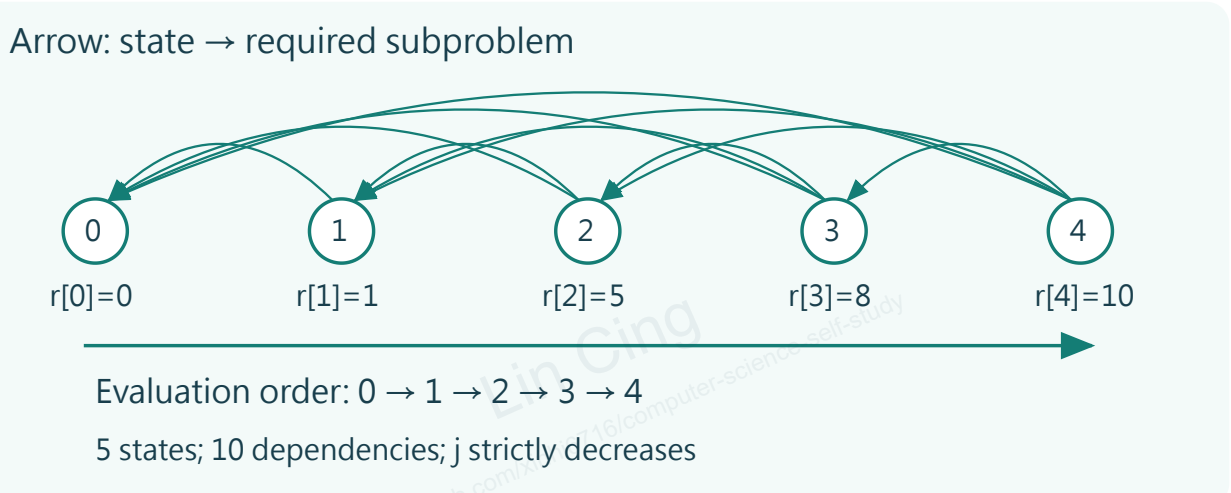
Together these give equality. Replacing the remainder preserves feasibility of the first piece: this is the essential optimal-substructure argument.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Overlap becomes a dependency DAG

An arrow $j \to k$ means that evaluating $r[j]$ requires $r[k]$.




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Evaluate shorter dependencies before longer states

Every $k < j$ is a candidate dependency of $r[j]$. For example, both $r[3]$ and $r[4]$ require $r[2]$. Merging equal-length calls stores that answer once.

For length 4, the dependency DAG has 5 states, $0, 1, 2, 3, 4$. State $j$ has $j$ outgoing dependency edges, so the total is $1 + 2 + 3 + 4 = 10$.

Evaluate $0, 1, 2, 3, 4$, opposite to the dependency-arrow direction. When $r[j]$ is evaluated, every needed $r[k]$ is already complete. The strictly smaller dependency length prevents a cycle.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Memoization expands each state once

```
memo[0] = 0; all other entries are UNKNOWN
solve(j):
    if memo[j] is not UNKNOWN: return memo[j]
    q = -∞
    for i = 1 .. j:
        q = max(q, p[i] + solve(j-i))
    memo[j] = q
    return q
```

The target requests its dependencies recursively. Repeated requests return cached values. Use a separate `UNKNOWN` marker; a negative revenue can be a valid answer.

Every length is reachable in this problem: time $\Theta(n^2)$, table space $\Theta(n)$, and at most $O(n)$ recursive stack space.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Tabulation also records the first piece

```
r[0] = 0
for j = 1 .. n:
    r[j] = -∞
    for i = 1 .. j:           # increasing i
        q = p[i] + r[j-i]
        if q > r[j]:         # ties retain smaller first piece
            r[j] = q
            first[j] = i
```

Loop invariant: when length $j$ starts, $r[0..j-1]$ are correct. Every candidate reads only completed entries.

Time is $\sum_{j=1}^{n} j = n(n+1)/2 = \Theta(n^2)$. Value and choice arrays together take $\Theta(n)$ space.



## A complete candidate trace

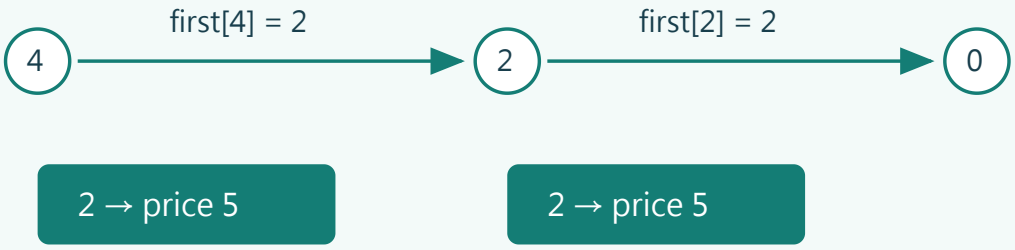
List $p[i] + r[j-i]$ in increasing order $i = 1, 2, \ldots, j$.

| $j$ | Candidate revenues, increasing $i$ | Running maximum | $r[j]$ | first[$j$] |
|---|---|---|---|---|
| 0 | Empty decomposition | — | 0 | — |
| 1 | $1+0=1$ | 1 | 1 | 1 |
| 2 | $1+1=2,\ 5+0=5$ | 2, 5 | 5 | 2 |
| 3 | $1+5=6,\ 5+1=6,\ 8+0=8$ | 6, 6, 8 | 8 | 3 |
| 4 | $1+8=9,\ 5+5=10,\ 8+1=9,\ 9+0=9$ | 9, 10, 10, 10 | 10 | 2 |

The highest individual selling price need not be the best first piece: $p[4] = 9$, while two pieces of length 2 earn 10.

## Follow recorded choices to recover the optimal cuts

Reconstruction: read first, then shorten the remainder




Output [2, 2]; total length 4; revenue 10

Stop at j = 0; never read first[0]

Here $first[4] = 2$ and $first[2] = 2$. The remaining length follows $4 \to 2 \to 0$, outputting $[2, 2]$. Check length $2 + 2 = 4$ and revenue $5 + 5 = 10 = r[4]$.



## Why recorded cuts preserve the optimum

At remaining length $j$, the emitted pieces have total length $n - j$, and their revenue plus $r[j]$ equals $r[n]$. Initially $j = n$ and no pieces have been emitted, so both claims hold.

Let $i = first[j]$. Emitting $i$ and replacing $j$ by $j - i$ increases emitted length by $i$. Because $r[j] = p[i] + r[j - i]$, the emitted revenue rises by exactly the amount removed from the remaining optimal revenue. Both claims are preserved.

At $j = 0$, the emitted length is $n$ and $r[0] = 0$, so emitted revenue is $r[n]$. In the example, emitting 2 twice gives total length 4 and revenue 10.

## Reconstruct until the remaining length is zero

After tabulation, `first[j]` records an optimal first piece for every positive length $j$. Follow those choices to produce the complete list.

```
j = n; pieces = []
while j > 0:
    i = first[j]
    append i to pieces
    j = j - i
return pieces
```

Every choice satisfies $1 \le first[j] \le j$. Each step reduces $j$ by at least 1, so the loop stops at zero after at most $n$ iterations and takes $O(n)$ time. The invariant then gives total length $n$ and revenue $r[n]$.

For $n = 0$, the loop does not run and returns an empty list.



## Two schedules for the same dependency graph

| Aspect | Top-down memoization | Bottom-up tabulation |
|---|---|---|
| Start | Request dependencies from the target | Evaluate states in a valid order |
| States evaluated | Reachable states only | Usually a prescribed range |
| Extra overhead | Calls and cache lookups | Usually no recursive stack |
| This example: time / table | $\Theta(n^2)$ / $\Theta(n)$ | $\Theta(n^2)$ / $\Theta(n)$ |

Memoization is not automatically faster; tabulation need not use a rectangular table. Both need a correct recurrence and an acyclic dependency schedule.

## Boundary conditions change the problem

**Use the entire rod:** For $p[1] = -2, p[2] = -5, r[1] = -2$ and $r[2] = \max(-4, -5) = -4$. Initialize each maximum to $-\infty$, not zero.

**Allow discarded material:** Change the recurrence to $r[j] = \max(0, \max_i(p[i] + r[j - i]))$. The same negative-price example now has value 0. Reconstruction must also record "stop and discard."

**Empty input:** With $n = 0$, prices may be empty; revenue is 0 and the decomposition is empty. For $n > 0$, this chapter assumes a price for every length $1..n$; a missing price is not zero.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Recheck the recurrence when assumptions change

Suppose each actual cut costs $c \geq 0$. Selling the entire remaining rod involves no cut.

$$r[j] = \max_{1 \leq i \leq j} \left( p[i] + r[j - i] - c\, \mathbf{1}_{i<j} \right).$$

For the original prices with $c = 2$, two pieces of length 2 earn $5 + 5 - 2 = 8$; selling length 4 uncut earns 9 and is now optimal.

Length still suffices as a state. Piece-count constraints or history-dependent prices may require more dimensions: check whether state alone determines future feasibility and cost.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Five checks for a DP design

1. Define the feasible solution set of each state and its bases.
2. Choose a first or last decision and enumerate every feasible candidate.
3. Prove that candidates cover an optimum and compose into feasible solutions.
4. Find a strictly decreasing measure and a valid evaluation order.
5. Count states and candidate work; verify reconstruction separately.

For rod cutting: length states, a first-piece decision, a two-direction proof, decreasing length, and quadratic time. Validate the output by checking both total length and revenue.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 0/1 knapsack: one copy per item

There are $n$ items with positive integer weights $w_i$, values $v_i$, and a nonnegative integer capacity $W$.

$$\max \sum_{i=1}^{n} v_i x_i \quad \text{subject to} \quad \sum_{i=1}^{n} w_i x_i \le W, \quad x_i \in \{0, 1\}.$$

The empty set is allowed, so the optimum is at least zero; negative-value items can be omitted. Equal weights or values do not make two items the same item. Integer capacities support this array-indexed DP. Arbitrary real weights cannot directly index the table.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Why the state includes an item prefix

Let $D[i, c]$ be the maximum value using only items $1, \ldots, i$ with total weight at most $c$. The answer is $D[n, W]$. Remaining capacity alone is insufficient: different available items can give different answers at the same capacity. A prefix identifies the available set and prevents reuse of an item. For a fixed $(i, c)$, the available choices depend only on that prefix and capacity. Of histories reaching the same state, an inferior value cannot improve an extension. This assumes additive values and no constraints linking items.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Two disjoint cases give the recurrence

$$D[0, c] = 0, \qquad D[i, 0] = 0.$$

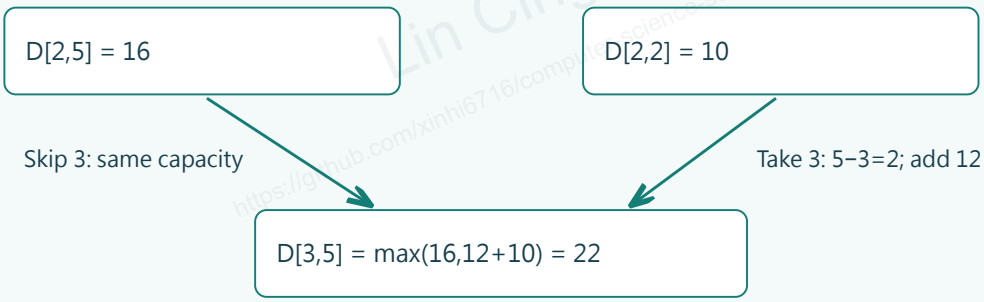
If $c < w_i$, then $D[i, c] = D[i-1, c]$. Otherwise,

$$D[i, c] = \max\{D[i-1, c],\ v_i + D[i-1, c - w_i]\}.$$

Skipping $i$ preserves capacity. Taking $i$ consumes $w_i$. Both cases leave only the first $i - 1$ items.

Two previous-row dependencies of one state

i = 3, c = 5, $(w_3, v_3) = (3, 12)$




Arrows = data flow; both cases reduce the item prefix from 3 to 2

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Proving both inequalities

Consider any feasible set $S$ for $(i, c)$. If $i \notin S$, its value is at most $D[i-1, c]$. If $i \in S$, removing $i$ leaves a set among the first $i-1$ items of weight at most $c - w_i$, so its value is at most $v_i + D[i-1, c-w_i]$. Thus the optimum is at most the maximum candidate. Conversely, an optimal set for the skip case is feasible unchanged. An optimal set for the take case excludes $i$; adding $i$ preserves feasibility. Both candidates are attainable, so their maximum is at most the optimum. Together with the base case, induction on $i$ proves correctness. Equivalently, if the remainder after removing $i$ were inferior, replacing it and adding $i$ back would contradict optimality.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## One example: inputs and table order

Capacity $W = 5$; item pairs $(w_i, v_i)$ are $(2, 10), (1, 6), (3, 12)$.

Rows are available prefixes and columns are capacity limits, not required exact weights.

Each cell reads only the previous row. Fill rows in order $i = 1, 2, 3$; capacities within a row may be visited in any order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## One example, the complete table

Complete table and traceback: W = 5

Items (w,v): 1=(2,10), 2=(1,6), 3=(3,12)

| Prefix i / capacity c | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| i = 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| i = 1 | 0 | 0 | 10 | 10 | 10 | 10 |
| i = 2 | 0 | 6 | 10 | 16 | 16 | 16 |
| i = 3 | 0 | 6 | 10 | 16 | 18 | 22 |

Arrows = traceback: take 3 → skip 2 → take 1; weight 5, value 22

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Reading the comparisons

The completed table gives $D[3,5] = 22$.

Item 1: $D[1,1] = 0$ (too heavy), and $D[1,2] = \max(0, 10+0) = 10$. Item 2: $D[2,3] = \max(D[1,3], 6 + D[1,2]) = \max(10, 16) = 16$. Item 3: $D[3,3] = \max(16, 12 + D[2,0]) = 16$: fitting does not force selection. Also $D[3,4] = \max(16, 12+6) = 18$ and $D[3,5] = \max(16, 12+10) = 22$. A take candidate adds the current item's value to a feasible optimum for the specified prefix and residual capacity, not to an arbitrary maximum in the same row.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Traceback recovers the chosen items

Start at $(i, c) = (n, W)$. If $D[i, c] = D[i-1, c]$, skip $i$ and decrement $i$. Otherwise take $i$, set $c \leftarrow c - w_i$, and decrement $i$. On equality, consistently skip to return one optimum. Here $(3, 5) : 22 > 16$, so take 3 and move to $(2, 2)$. Since $10 = 10$, skip 2 and move to $(1, 2)$. Since $10 > 0$, take 1 and reach $(0, 0)$. The result is $\{1, 3\}$, weight $2 + 3 = 5$, value $10 + 12 = 22$. Each step decreases $i$, giving $O(n)$ traceback. Capacity decrease alone is not the loop invariant.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A complete two-dimensional algorithm

```
D[0..n][0..W] = 0
for i = 1..n:
    for c = 0..W:
        D[i,c] = D[i-1,c]
        if c >= w[i]:
            D[i,c] = max(D[i,c], v[i] + D[i-1,c-w[i]])
```

There are $(n + 1)(W + 1)$ cells and constant work per cell. Time is $O(n(W + 1))$ and full-table space is $O(n(W + 1))$, conventionally $O(nW)$ when $W \geq 1$. The retained table supports the preceding traceback.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## One dimension: descend to preserve the old row

```
dp[0..W] = 0
for i = 1..n:
    for c = W down to w[i]:
        dp[c] = max(dp[c], v[i] + dp[c-w[i]])
```

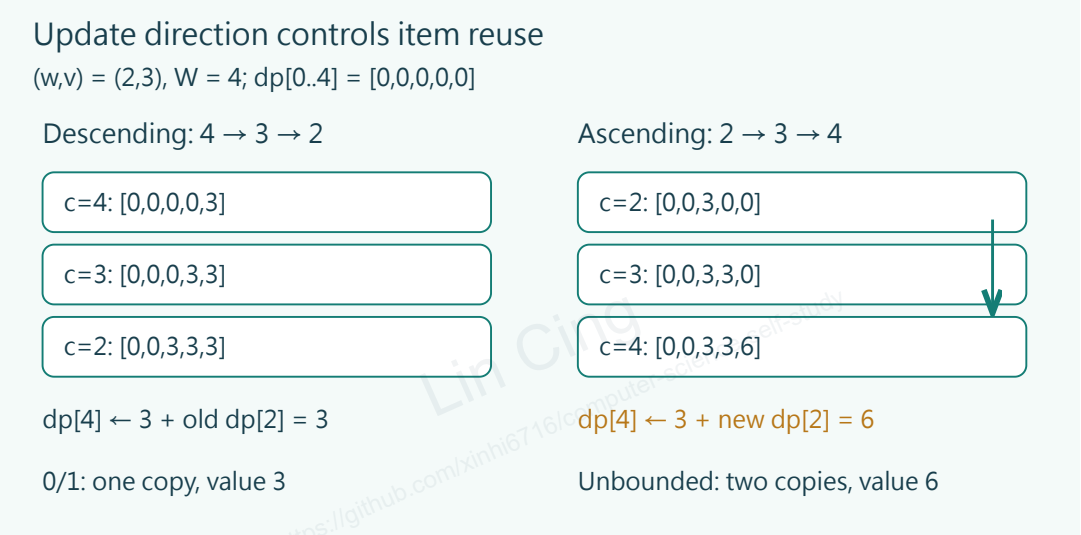
During item $i$, unmodified capacities hold $D[i-1,\cdot]$. Descending order and $w_i > 0$ ensure $c - w_i < c$ is still unmodified. The right side reads the previous row; afterward $dp[c] = D[i,c]$. Time remains $O(nW)$; value storage falls to $O(W)$. Overwritten history does not directly support two-dimensional traceback. To recover items, retain the full table or use versioned decision records.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Ascending order reuses the same item

There is one item $(w,v) = (2,3)$ and capacity $W = 4$. The 0/1 optimum must be 3.




Ascending updates first write $dp[2] = 3$. At $c = 4$, the new $dp[2]$ yields 6, equivalent to taking two copies of the single item. Loop direction selects the version of the state being read.

## Unbounded knapsack needs the current row

If each item type has arbitrarily many copies, let $U[i, c]$ be the maximum value using the first $i$ types within capacity $c$. When the item fits,

$$U[i, c] = \max\{U[i-1, c],\ v_i + U[i, c - w_i]\}.$$

After taking a copy, type $i$ remains available, so the second term uses the same row. An ascending capacity loop implements this dependency in one dimension. Since $w_i > 0$, the same-row dependency has strictly smaller capacity. For $(2, 3), W = 4$, two copies give 6, so the preceding ascending result matches this model. Zero-weight items are excluded here: unlimited copies of a zero-weight positive-value type make the objective unbounded.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Exact weight: infeasible does not mean zero

Let $E[i, c]$ be the maximum value using the first $i$ items with total weight **exactly** $c$. Use the same 0/1 recurrence, but initialize

$$E[0, 0] = 0, \quad E[0, c] = -\infty\ (c > 0).$$

An infeasible candidate stays $-\infty$. Check reachability before addition to avoid overflow from integer sentinels. Positive weights imply $E[i, 0] = 0$. For a single item $(2, -3), W = 2$, the at-most optimum is 0 (empty), whereas the exact-weight optimum is $-3$. For $W = 1$, exact weight is infeasible. Initializing every cell to zero invents feasible weights.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Why the bound is pseudopolynomial

$O(nW)$ is polynomial in the numerical capacity $W$, but not necessarily in binary input length. Encoding capacity uses about $\log_2(W+1)$ bits; if $W = 2^b$, the capacity loop takes $2^b$ steps. For 20 items, capacity $10^3$ means about 20,000 updates; capacity $10^9$ means about 20 billion. Indexable integer weights do not guarantee a practical table. This analysis treats each value addition and comparison as constant cost. Very large integer values also require arithmetic bit costs. The decision version of 0/1 knapsack is NP-complete; the optimization version is NP-hard.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Why fractional knapsack admits greedy choice

With $0 \le x_i \le 1$, items may be split. Process positive-value items in decreasing density $v_i/w_i$; omit negative values. If lower-density weight is selected while higher-density supply remains, exchanging equal weight $\delta$ cannot decrease value. Repeated exchanges yield the density-greedy solution. Use the same items with capacity changed to $W = 4$. Densities are 6 for item 2, 5 for item 1, and 4 for item 3. The fractional optimum takes 2, 1, and $1/3$ of 3, giving $6 + 10 + 4 = 20$. The 0/1 density rule takes 2 and 1 for 16; the 0/1 optimum takes 2 and 3 for 18. Equal-weight exchanges may be impossible without splitting. Sorting gives $O(n \log n)$ time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A branch-and-bound bound is not a DP value

For maximization, branch and bound explores a take/skip search tree while keeping an incumbent feasible value $L$. Let $V$ be the value already taken and $C$ the remaining capacity. Let $F(C)$ be the fractional optimum using only undecided items. The total upper bound at the node is $B = V + F(C)$. Relaxation enlarges the feasible set, so every 0/1 completion at the node has value at most $B$. If $B \leq L$, prune the node when only one optimum is required. In the previous example, the root has $B = 20$ while the integer optimum is 18: a bound need not be attainable. A DP entry $D[i, c]$ is the **exact** optimum of its defined subproblem. These quantities serve different purposes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Subsequences preserve order, not adjacency

A subsequence uses strictly increasing indices. A substring must also be contiguous.

Fix $X =$ `ABCBDAB` and $Y =$ `BDCABA`. `BCBA` uses indices $(2, 3, 4, 6)$ in $X$ and $(1, 3, 5, 6)$ in $Y$. Its length is 4. `BCA` is common but is not longest.

First compute the **length** of a longest common subsequence (LCS); then reconstruct one witness.

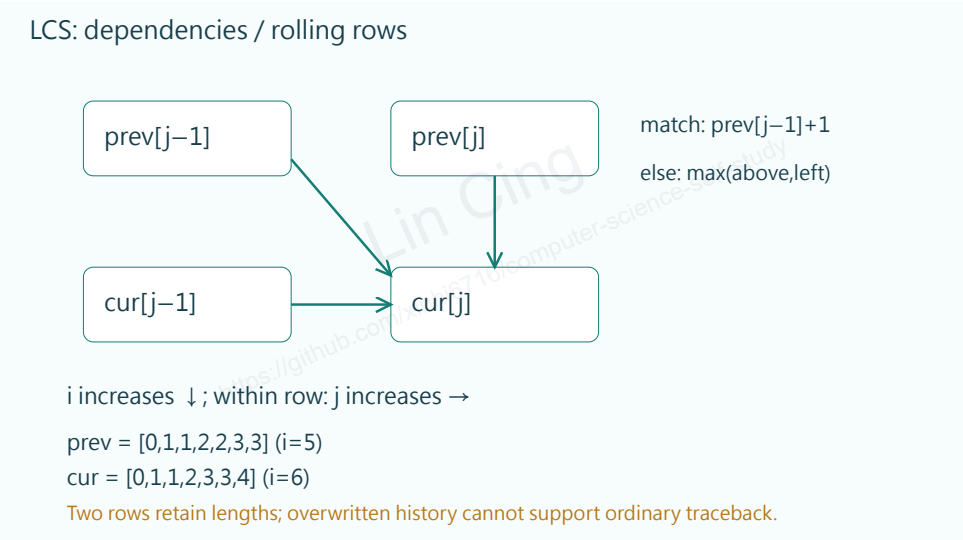
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A state is a pair of prefixes

Let $C[i, j]$ be the LCS length of $X[1..i]$ and $Y[1..j]$. The answer is $C[7, 6]$.

$C[0, j] = C[i, 0] = 0$: an empty prefix has only the empty subsequence.

Dependencies decrease $i + j$, forming a DAG. Fill rows $i = 1..m$, scanning $j = 1..n$ within each row. Above, left, and diagonal predecessors are then ready.






## Equal final symbols: an achievable lower bound

If $x_i = y_j = a$, append $a$ to an LCS of the two shorter prefixes. The resulting indices are legal, so

$$C[i, j] \geq C[i - 1, j - 1] + 1.$$

This constructs a feasible candidate. At $(6, 6)$ both final symbols are A and $C[5, 5] = 3$, so length 4 is achievable. A lower bound alone does not prove optimality.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Equal final symbols: the matching upper bound

Take any LCS $Z$ of length $k$. Its last symbol must be $a$: otherwise both final positions are unused, and appending $a$ would make a longer common subsequence.

Remove the last $a$ of $Z$. The remaining symbols embed in both shorter prefixes, hence $k - 1 \leq C[i-1, j-1]$. Combining both bounds gives

$$C[i, j] = C[i-1, j-1] + 1.$$

An optimal embedding using both final positions exists. This does not force every optimal index matching to use them: for A versus AA, either A position can be matched.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Unequal final symbols: prove both inequalities

If $x_i \neq y_j$, an embedding of a common subsequence cannot use both final positions: both would have to match its same last symbol.

Thus an optimal embedding omits $x_i$ or $y_j$, so its length is at most $\max(C[i-1, j], C[i, j-1])$.

Conversely, every common subsequence of either shorter-prefix pair remains feasible in the full pair. Choosing the longer candidate attains that bound. Therefore

$$C[i, j] = \max\{C[i-1, j], C[i, j-1]\}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## The complete length table

Equal symbols use diagonal +1; unequal symbols use max(above, left). Keep the zero boundary.

LCS: ABCBDAB / BDCABA

row i: X prefix; column j: Y prefix

| | 0:ø | 1:B | 2:D | 3:C | 4:A | 5:B | 6:A |
|---|---|---|---|---|---|---|---|
| 0:ø | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1:A | 0 | 0 | 0 | 0 | 1 | 1 | 1 |
| 2:B | 0 | 1 | 1 | 1 | 1 | 2 | 2 |
| 3:C | 0 | 1 | 1 | 2 | 2 | 2 | 2 |
| 4:B | 0 | 1 | 1 | 2 | 2 | 3 | 3 |
| 5:D | 0 | 1 | 2 | 2 | 2 | 3 | 3 |
| 6:A | 0 | 1 | 2 | 2 | 3 | 3 | 4 |
| 7:B | 0 | 1 | 2 | 2 | 3 | 4 | 4 |

Arrows: traceback to predecessor; orange circles: step; shading: chosen path

Output BCBA; length 4

$C[3,3] = C[2,2] + 1 = 2$; $C[5,5] = \max(3,2) = 3$; $C[7,6] = \max(4,4) = 4$. Numbered arrows identify the exact traceback on the next card.



## Traceback: reverse ABCB to obtain BCBA

For unequal symbols, prefer above when above and left tie. For equal symbols, move diagonally and record the symbol.

| Step | Current → predecessor | Reason / record |
|---|---|---|
| 1 | $(7,6) \to (6,6)$ | both 4; prefer above |
| 2 | $(6,6) \to (5,5)$ | A=A; record A |
| 3 | $(5,5) \to (4,5)$ | above 3 > left 2 |
| 4 | $(4,5) \to (3,4)$ | B=B; record B |
| 5 | $(3,4) \to (3,3)$ | left 2 > above 1 |
| 6 | $(3,3) \to (2,2)$ | C=C; record C |
| 7 | $(2,2) \to (2,1)$ | left 1 > above 0 |
| 8 | $(2,1) \to (1,0)$ | B=B; record B; stop |

## Ties choose a witness, not its length

Recorded backward: ABCB. Reverse it to obtain BCBA.

BCBA, BCAB, and BDAB are all LCSs of length 4. A different tie rule may return another string without changing the optimum length.

At $(7, 6)$, moving left to $(7, 5)$ is also optimal. Preferring above simply makes this example deterministic.

One stored predecessor reconstructs one answer. Enumerating all distinct LCSs requires exploring all optimal branches and removing duplicates. The number of traceback paths need not equal the number of distinct strings.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Computing a value and rebuilding a witness

The table has $(m + 1)(n + 1)$ states and constant work per interior cell: $\Theta(mn)$ time and $\Theta(mn)$ space.

Traceback decreases at least one index at every step, taking $O(m + n)$ time. Store a choice table, or keep the full value table and compare predecessors during traceback.

Brute-force index-subset enumeration has $2^m$ candidates on one side. Many recursive routes ask for the same prefix pair. DP merges those repeated states.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Rolling rows preserve lengths, not the history

Use the shorter string for columns. `prev` is the completed preceding row; `cur` is the row being filled from left to right.

```
prev[0..n] = 0
cur[0..n] = 0
for i = 1..m:
    cur[0] = 0
    for j = 1..n:
        if X[i] == Y[j]: cur[j] = prev[j-1] + 1
        else: cur[j] = max(prev[j], cur[j-1])
    swap(prev, cur)
return prev[n]
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Rolling rows and witness reconstruction

Time remains $\Theta(mn)$; auxiliary space becomes $O(\min(m, n))$. This code returns only a length.

Each update reads the previous row and the current row's left neighbor. Once the rows swap, the old history can be overwritten because later length calculations no longer need it.

Overwritten rows no longer support ordinary full-table traceback. To output a sequence, retain the full value/choice table or use another reconstruction algorithm, such as divide and conquer.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Global alignment accounts for every symbol

Fix $X = \texttt{ACGTA}$ and $Y = \texttt{AGCAC}$. Insert gaps `-` to make equally long rows. A `-/-` column is forbidden. Removing gaps must recover the original strings.

| Column type | Score |
|---|---|
| matching symbols | +3 |
| mismatching symbols | −2 |
| one symbol and one gap | −3 |

Maximize the sum of all column scores, including leading and trailing gaps. This is global alignment with a linear gap cost. Its score is not an LCS length.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Alignment state and complete initialization

$F[i, j]$ is the best global alignment score of prefixes $X[1..i]$ and $Y[1..j]$.

$$F[0, 0] = 0, \qquad F[i, 0] = -3i, \qquad F[0, j] = -3j.$$

If only one prefix is nonempty, each symbol must face a gap. Both boundaries are `0, -3, -6, -9, -12, -15`.

Zero boundaries would allow an unmatched prefix for free and change the problem. Adding a zero candidate also changes the specified global-alignment recurrence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## The final column has three possible forms

Let $s(a, b) = 3$ for equal symbols and $-2$ otherwise. For $i, j > 0$:

$$F[i,j] = \max \begin{cases} F[i-1, j-1] + s(x_i, y_j) & \text{symbol / symbol} \\ F[i-1, j] - 3 & x_i \text{ / gap} \\ F[i, j-1] - 3 & \text{gap / } y_j. \end{cases}$$

Every legal alignment ends in one of these forms. Removing its last column leaves the corresponding prefix alignment, bounding its score by the maximum above. Conversely, append each listed column to an optimal predecessor alignment to attain that candidate. Both inequalities establish the recurrence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A complete alignment table: optimum 1

Global alignment: ACGTA / AGCAC

row i: X prefix; column j: Y prefix

| | 0:ø | 1:A | 2:G | 3:C | 4:A | 5:C |
|---|---|---|---|---|---|---|
| 0:ø | 0 | -3 | -6 | -9 | -12 | -15 |
| 1:A | -3 | 3 | 0 | -3 | -6 | -9 |
| 2:C | -6 | 0 | 1 | 3 | 0 | -3 |
| 3:G | -9 | -3 | 3 | 0 | 1 | -2 |
| 4:T | -12 | -6 | 0 | 1 | -2 | -1 |
| 5:A | -15 | -9 | -3 | -2 | 4 | 1 |

Arrows: traceback to predecessor; orange circles: step; shading: chosen path

Output ACGTA- / A-GCAC; total score 1

$F[2,3] = \max(0+3, -3-3, 1-3) = 3.$

## Reconstruct every alignment column

Trace backward from $(5, 5)$:

| Current → predecessor | Column | Added score |
|---|---|---|
| $(5, 5) \to (5, 4)$ | `-/C` | −3 |
| $(5, 4) \to (4, 3)$ | `A/A` | +3 |
| $(4, 3) \to (3, 2)$ | `T/C` | −2 |
| $(3, 2) \to (2, 1)$ | `G/G` | +3 |
| $(2, 1) \to (1, 1)$ | `C/-` | −3 |
| $(1, 1) \to (0, 0)$ | `A/A` | +3 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Read the reconstructed alignment forward

Reverse the column order:

```
ACGTA-
A-GCAC
```

Total score: $3 - 3 + 3 - 2 + 3 - 3 = 1$. Both original sequences are fully represented.

The two gap columns account for C in $X$ and the final C in $Y$. The T/C mismatch contributes $-2$; the three matching columns contribute $3 + 3 + 3$.

$F[5, 5] = \max(-2 - 2, -1 - 3, 4 - 3) = 1$, so its final column is `-/C`. Interior scores may be negative even when the final score is positive; global alignment does not reset them to zero.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Similar grids, different meanings

| Problem | State value | Equal symbols | Unequal symbols | Boundary |
|---|---|---|---|---|
| LCS | maximum length | diagonal +1 | max(above,left) | 0 |
| Global alignment | maximum total score | compare all 3 candidates | compare all 3 candidates | accumulated gap cost |

LCS skips symbols for free. Global alignment must place every symbol in a scored column. Even when symbols match, the alignment algorithm compares its three specified branches; the LCS if/else is not its recurrence.

Both use a prefix DAG, and both require enough predecessor information to reconstruct a witness.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Interval DP: the final decision

The last matrix multiplication chooses a split $k$; a BST root $r$ determines the left and right key intervals. Both preserve the original order.

A state $[i, j]$ represents a contiguous interval. Fill increasing lengths because every decision depends on shorter intervals. The value table answers “how good”; the decision table answers “how to build”.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Matrix chains: dimensions determine merge cost

Let $p = [5, 10, 3, 12, 5]$. Matrix $A_i$ has dimensions $p_{i-1} \times p_i$.

| Matrix | $A_1$ | $A_2$ | $A_3$ | $A_4$ |
|---|---|---|---|---|
| Dimensions | $5 \times 10$ | $10 \times 3$ | $3 \times 12$ | $12 \times 5$ |

Under standard multiplication, $(a \times b)(b \times c)$ costs $abc$ scalar multiplications and produces $a \times c$. Change parentheses, preserving order. Planning does not perform the products.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



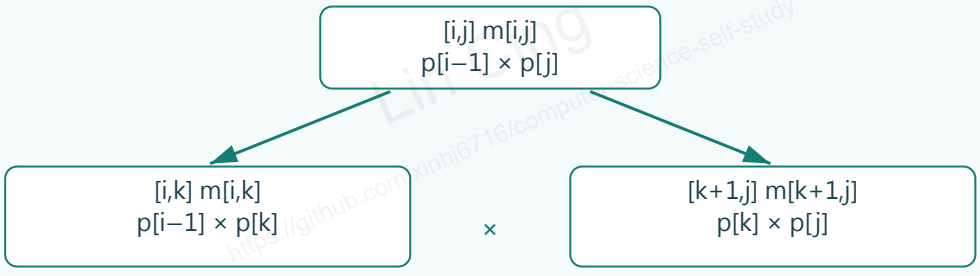
## Matrix-chain recurrence and sufficient state

Let $m[i, j]$ be the minimum multiplication count for $A_i \cdots A_j$, with $m[i, i] = 0$.

$$m[i, j] = \min_{i \le k < j} \{m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j\}.$$

Every parenthesization of this interval produces $p_{i-1} \times p_j$. The outside computation needs endpoints and minimum cost, without the internal parentheses. A fixed split fixes both child dimensions.

### Fixed split: child dimensions survive replacement




merge = p[i−1] · p[k] · p[j]

Arrows: parent depends on shorter intervals; replacement lowers child cost

## Proof by splitting, replacing, and combining

Every full parenthesization has a final split $i \leq k < j$. If its left child is suboptimal, replace it with a cheaper solution of the same output dimensions. The right cost and merge cost stay fixed, reducing the total: a contradiction. The right child follows identically.

Conversely, for every split, two feasible child products can be multiplied legally. Each candidate is a feasible complete solution. Candidates include an optimum and introduce no invalid solution; their minimum is optimal. Induction on interval length establishes every table entry.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Fill by length: all short-interval candidates

Initialize every diagonal $m[i, i] = 0$. Each candidate is "left + right + merge".

| Length | Interval | Candidates and arithmetic | $m$; $s$ |
|---|---|---|---|
| 2 | $[1, 2]$ | $k = 1 : 0 + 0 + 5 \cdot 10 \cdot 3 = 150$ | 150; 1 |
| 2 | $[2, 3]$ | $k = 2 : 0 + 0 + 10 \cdot 3 \cdot 12 = 360$ | 360; 2 |
| 2 | $[3, 4]$ | $k = 3 : 0 + 0 + 3 \cdot 12 \cdot 5 = 180$ | 180; 3 |
| 3 | $[1, 3]$ | $k = 1 : 0 + 360 + 600 = 960$; $k = 2 : 150 + 0 + 180 = 330$ | 330; 2 |
| 3 | $[2, 4]$ | $k = 2 : 0 + 180 + 150 = 330$; $k = 3 : 360 + 0 + 600 = 960$ | 330; 2 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## The full chain: compare all three splits

| Split for $[1, 4]$ | Left | Right | Final multiplication | Total |
|---|---|---|---|---|
| $k = 1$ | 0 | 330 | $5 \cdot 10 \cdot 5 = 250$ | 580 |
| $k = 2$ | 150 | 180 | $5 \cdot 3 \cdot 5 = 75$ | **405** |
| $k = 3$ | 330 | 0 | $5 \cdot 12 \cdot 5 = 300$ | 630 |

The merge cost alone is not a general greedy rule: both optimal child costs must be included. In this example, $k = 2$ happens to minimize both merge cost and total cost.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## The full chain: compare all three splits (continued)

### Complete matrix-chain table: m[i,j] / s[i,j]

| | j=1 | j=2 | j=3 | j=4 |
|---|---|---|---|---|
| i=1 | 0 / – | 150 / 1 | 330 / 2 | 405 / 2 |
| i=2 | | 0 / – | 360 / 2 | 330 / 2 |
| i=3 | | | 0 / – | 180 / 3 |
| i=4 | | | | 0 / – |

Fill lengths L=1 → 2 → 3 → 4; read only shorter intervals

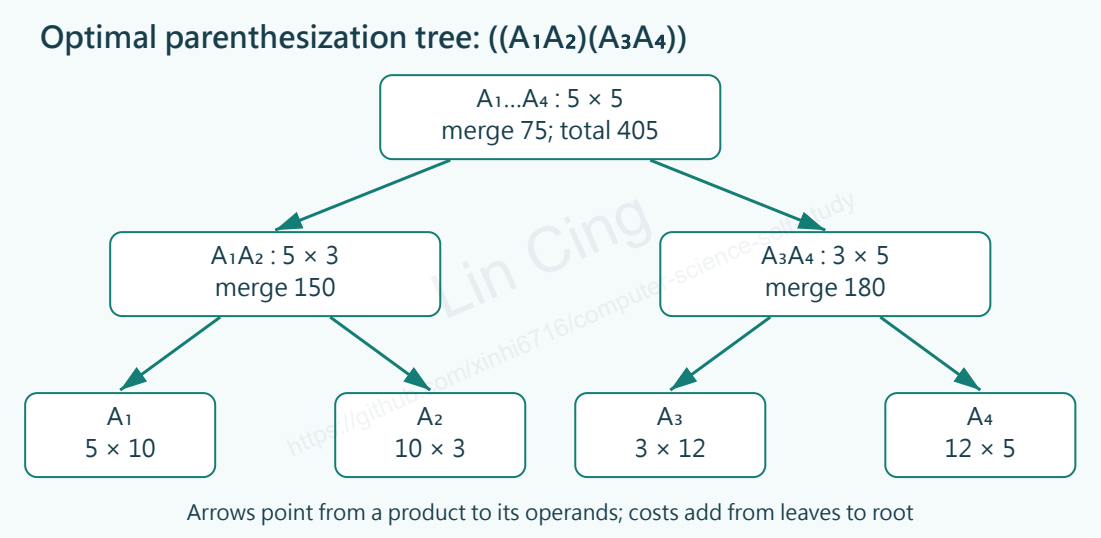
**L=4: 580, 405, 630 → min = 405**

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Reconstruct the parenthesization tree

Use $s[1,4]=2$, then $s[1,2]=1$ and $s[3,4]=3$, yielding $((A_1A_2)(A_3A_4))$.

Compute $A_1A_2 : 5\times 3$ at cost 150 and $A_3A_4 : 3\times 5$ at cost 180. Their product is $5\times 5$ at cost 75; total 405.




Recursion: for $i=j$, output $A_i$. Otherwise output parentheses around `build(i,s[i,j])` and `build(s[i,j]+1,j)`. Each leaf and merge node is visited once: $\Theta(n)$ reconstruction.



## Matrix-chain algorithm and exact work

```
for i=1..n: m[i,i]=0
for L=2..n:
  for i=1..n-L+1:
    j=i+L-1; m[i,j]=infinity
    for k=i..j-1:
      t=m[i,k]+m[k+1,j]+p[i-1]*p[k]*p[j]
      if t<m[i,j]: m[i,j]=t; s[i,j]=k
```

Length $L$ has $n-L+1$ states and $L-1$ candidates per state:

$$\sum_{L=2}^{n}(n-L+1)(L-1)=\binom{n+1}{3}=\frac{n^3-n}{6}.$$

Time is $\Theta(n^3)$; value and split tables each occupy $\Theta(n^2)$ space. These bounds concern planning, not executing matrix products.

## OBST: model successful and unsuccessful searches

Sorted real keys are $k_1 < \cdots < k_n$, with successful-search weights $p_1, \ldots, p_n$. External leaves $d_0, \ldots, d_n$ represent unsuccessful-search gaps, with weights $q_0, \ldots, q_n$.

$d_0$ means below $k_1$; $d_i$ means $k_i < x < k_{i+1}$; $d_n$ means above $k_n$. An extended BST with $n$ real keys has $n + 1$ external leaves.

Probabilities satisfy $\sum p + \sum q = 1$. Nonnegative frequency weights also work directly; divide total cost by the global total weight to obtain expected cost per query.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Two cost conventions: a constant difference

The root has depth 0. $C(T)$ counts real-key comparisons; $E(T)$ also counts one terminal check at the external leaf.

$$C(T) = \sum_i p_i(\text{depth}(k_i) + 1) + \sum_j q_j \,\text{depth}(d_j).$$

$$E(T) = \sum_i p_i(\text{depth}(k_i) + 1) + \sum_j q_j(\text{depth}(d_j) + 1).$$

Thus $E(T) - C(T) = \sum_j q_j$, independent of shape, so both have the same optimal trees. An empty interval contains only $d_{i-1}$: $C[i, i-1] = 0$ and $E[i, i-1] = q_{i-1}$. Initialization and reported costs must use the same convention.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## OBST intervals retain global probability mass

Interval $[i, j]$ contains keys $k_i, \ldots, k_j$ and external leaves $d_{i-1}, \ldots, d_j$.

$$w[i, j] = \sum_{t=i}^{j} p_t + \sum_{t=i-1}^{j} q_t.$$

Subproblems retain the original global weights, without dividing by interval mass. Lowering a subtree by one level adds one check for every query in it, increasing cost by global mass $w$.

If conditional expectations are stored instead, each must be multiplied by its subtree mass when combining. Conditional expectations with different denominators cannot simply be added. Empty intervals still have $w[i, i-1] = q_{i-1}$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## OBST recurrence and replacement proof

$$E[i, j] = w[i, j] + \min_{i \le r \le j} \{E[i, r-1] + E[r+1, j]\}.$$

$C$ has the same recurrence shape, with zero empty-interval costs. Inorder order uniquely determines both child key ranges; merging adds $w[i, j]$.

Every optimum has some root. Replace any suboptimal child with a cheaper tree over the same keys and gaps. BST order and subtree mass stay unchanged, as does the cost of placing it at the same depth. The total decreases, a contradiction. Conversely, every candidate root and two feasible children form a feasible BST. Minimizing over all roots is therefore optimal.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Three-key example B: initialization and singletons

Fix $p = (.20, .30, .10)$ and $q = (.10, .05, .15, .10)$, total mass 1.

Empty intervals: $E[1, 0] = .10$, $E[2, 1] = .05$, $E[3, 2] = .15$, $E[4, 3] = .10$.

| Interval | $w$ | Only candidate $E_L + E_R + w$ | $E$ | $C$ |
|---|---|---|---|---|
| $[1, 1]$ | .35 | $.10 + .05 + .35$ | .50 | .35 |
| $[2, 2]$ | .50 | $.05 + .15 + .50$ | .70 | .50 |
| $[3, 3]$ | .35 | $.15 + .10 + .35$ | .60 | .35 |

A singleton's $C$ includes unsuccessful comparisons for both adjacent gaps, so it is not just $p_i$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Example B: every root for pairs and the full interval

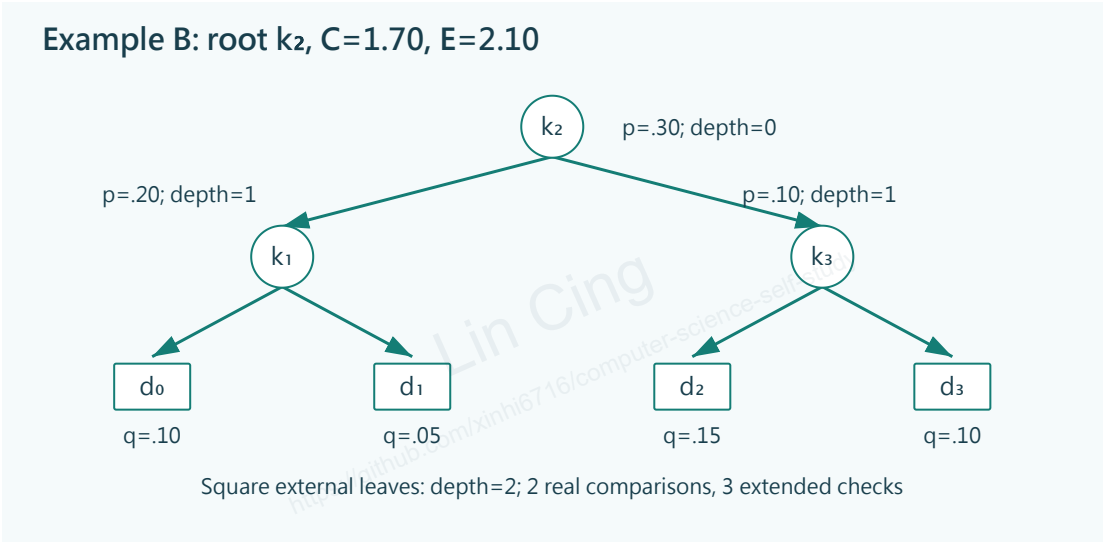
| Interval; mass | Candidate roots and complete arithmetic | Minimum $E$; $C$; root |
|---|---|---|
| $[1, 2]$; .80 | $r = 1 : .10 + .70 + .80 = 1.60$; $r = 2 : .50 + .15 + .80 = 1.45$ | 1.45; 1.15; 2 |
| $[2, 3]$; .70 | $r = 2 : .05 + .60 + .70 = 1.35$; $r = 3 : .70 + .10 + .70 = 1.50$ | 1.35; 1.05; 2 |
| $[1, 3]$; 1.00 | $r = 1 : .10 + 1.35 + 1 = 2.45$; $r = 2 : .50 + .60 + 1 = 2.10$; $r = 3 : 1.45 + .10 + 1 = 2.55$ | **2.10; 1.70; 2** |

Each interval has $C = E - \sum_{t=i-1}^{j} q_t$: subtract that interval's gap mass. The full table evaluates 10 root candidates, not just the final three.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Example B: reconstruct and audit the expectation

The root is $k_2$, with left child $k_1$ and right child $k_3$. Real-key depths are 1, 0, 1; all four external leaves have depth 2.






## Example B: account for each query outcome

The real-key comparison counts are 2, 1, and 2. Successful queries contribute $.20 \cdot 2 + .30 \cdot 1 + .10 \cdot 2 = .90$

The four unsuccessful gaps have total probability $.10 + .05 + .15 + .10 = .40$. Each unsuccessful query compares two real keys. Counting the external-leaf termination check raises its extended cost to three.

$C = .90 + (.10 + .05 + .15 + .10) \cdot 2 = 1.70$; $E = .90 + .40 \cdot 3 = 2.10$. The difference is .40. These are expectations per global query, not expectations conditional on success.

## OBST tabulation, reconstruction, and exact work

```
for i=1..n+1: E[i,i-1]=w[i,i-1]=q[i-1]
for L=1..n:
  for i=1..n-L+1:
    j=i+L-1; w[i,j]=w[i,j-1]+p[j]+q[j]
    E[i,j]=infinity
    for r=i..j:
      t=E[i,r-1]+E[r+1,j]+w[i,j]
      if t<E[i,j]: E[i,j]=t; root[i,j]=r
```

The candidate count is $\sum_{L=1}^{n}(n-L+1)L=\binom{n+2}{3}=n(n+1)(n+2)/6$. The basic algorithm takes $\Theta(n^3)$ time and $\Theta(n^2)$ space. `build(i,j)` returns $d_{i-1}$ when empty; otherwise attach both children to `root[i,j]`. Output contains $2n+1$ nodes and takes $\Theta(n)$ time.

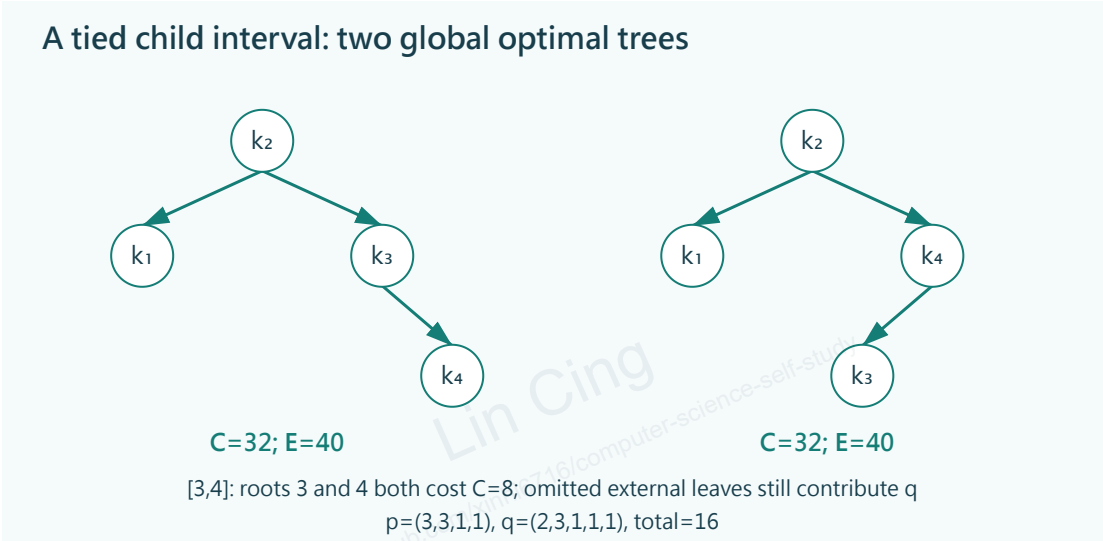


## Integer example C: two trees can share the optimum

Use frequency weights $p=(3,3,1,1)$, $q=(2,3,1,1,1)$, global total 16.

Interval $[3,4]$ has $w=5$. Its $C$ candidates are $0+3+5=8$ for root 3 and $3+0+5=8$ for root 4: a tie. The full root-2 candidate is $C[1,1]+C[3,4]+16=8+8+16=32$; roots 1, 3, 4 cost 35, 38, 41.

## Integer example C: two trees can share the optimum (continued)




Global $C = 32$, $E = 40$; dividing by 16 gives expectations 2 and 2.5. Storing the first minimizing root reconstructs one optimum. All optimal trees require every tied root, and output size may be exponential.



## Connecting successful-only and full models

| Model | Weights and boundaries | Objective |
|---|---|---|
| Data structures ch.10 successful-only version | Only $p$; empty cost 0; $W = \sum p$ | Weighted real-key comparisons for successful queries |
| Full model here | $p, q$; empty $C = 0$, empty $E = q$; $w = \sum p + \sum q$ | Global weighted cost of successful and unsuccessful queries |

The current data structures chapter also presents the full $p/q$ model and $E - C$ difference; successful-only is not its sole convention. The original data structures notes use real-key cost $C$; CLRS § 15.5 uses extended cost $E$.

Setting every $q$ to zero recovers successful-only. Deleting nonzero $q$ changes the problem.

## Checks that matter for interval DP

Matrix chains: endpoints fix output dimensions; use $i \leq k < j$; minimize the whole candidate cost; reconstruct recorded splits.

OBST: preserve key order and gap membership; align empty costs with the $C/E$ convention; retain global weights; compare every root; tie policy chooses among optimal trees.

Sharing an interval-DP pattern does not make merge costs or boundaries interchangeable. The analysis here concerns the basic cubic algorithms. Faster OBST optimization requires a separate root-monotonicity argument and does not automatically apply to arbitrary interval DP.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapter 15, Dynamic Programming. MIT 6.046J, Fall 2004, Lecture 12, Dynamic Programming. Lecturer Charles E. Leiserson. MIT 6.046J, Fall 2005, Lecture 15, Dynamic Programming. Lecturer Charles E. Leiserson; slides credited to Erik D. Demaine and Charles E. Leiserson. Original scanned algorithms notes (演算法_wjungle), dynamic programming pages; original scanned data structures notes (資料結構), physical pages 166–170, for comparison of optimal search tree models. These notes cover algorithms and data structures and are not the full text of a single textbook.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study